

The Performance Commons

Why ASIC-level performance is a collective-action problem, and the hardware we should build for its solution

A position paper. The underlying architecture analysis is drawn from the companion technical comparison, "Dataflow vs. an Optimal Heterogeneous Architecture."

Abstract

The reason we don't achieve near ASIC performance and power for many more applications (and the reason we don't have much better ASICs) is not because it's technically impossible given the engineering resources available. It is the collective-action problem both within and across levels of the computing stack. As machine intelligence continues to advance — as long as it maintains the apparent inoculation to this problem — this fact will increasingly be exposed. This raises many questions for forward looking engineers in the form of, "How do we design systems assuming the collective-action problem is solved?". For example, how do we design hardware assuming there will be orders of magnitude more intelligence available for shared code optimization? We likely want more reconfigurable and less "programmable" architectures. And similarly, how do we design software stacks assuming the programmability issues holding back wider adoption of distributed computing are solved?

1. The gap is real, and it is not physics

Start by splitting a concept the field treats as one thing. An architecture's "programmability" is two different numbers: the performance it yields at *constant, ordinary programmer effort* — what a competent team gets by default — and the best performance *achievable* on it, its **realizable frontier**, the ceiling the roofline model draws per kernel (Williams, Waterman & Patterson, CACM 2009). Comparing architectures to ASICs measures neither; the number that matters first is the distance between the two on a *fixed* architecture — the **realization gap** — and it is measured, on unchanged silicon. A 4096×4096 matrix multiply runs **more than 60,000× faster** hand-optimized than in naive Python, from 0.0006% to 41% of machine peak (Leiserson et al., Science 2020), and Intel measured a **"ninja gap" averaging 24×** between naively written and expert-optimized C++ on the same processor (Satish et al., ISCA 2012). Most general-purpose code runs one to two orders of magnitude below its own architecture's frontier — full HPC applications, already tuned harder than most software, measure at 4–14% of peak on commodity platforms (Oliker et al., SC 2004).

Nor is the frontier the industry does reach optimal. The M1 matched the best x86 single-thread performance at roughly a fifth the power — the same general-purpose programmability, strictly better, no tradeoff paid (AnandTech, 2020); Etched claims the same inference function at under half the industry's voltage for multiples of the FLOPs density (vendor claims; §5 tells both stories in full). A design point that can be dominated without surrendering programmability was never on the true frontier at all.

What physics actually charges for generality is a separate and smaller number, also measured: a general-purpose core is roughly **100–500× less energy-efficient than an ASIC** for the same task (Hameed et al., ISCA 2010); word-level reconfigurable hardware (a CGRA) sits within ~2.8× of an ASIC; bit-level reconfigurable hardware (an FPGA) ~20–35× off (Prabhakar et al., ISCA 2017; Kuon & Rose, 2007); a GPU lands roughly 3–15× off a node-normalized ASIC in energy on the regular data-parallel kernels it is built

for (Chung et al., MICRO 2010), and Google's first TPU delivered ~30× the performance-per-watt of its contemporary, pre-tensor-core GPU on datacenter inference (Jouppi et al., ISCA 2017). That GPU tax is collapsing where instructions amortize: with tensor-core matrix instructions, 77–87% of the energy goes to the arithmetic itself, bounding a dedicated accelerator's remaining advantage on the core matrix multiply at 13–23% — near-ASIC efficiency for the work the GPU specializes (Dally, Turakhia & Han, CACM 2020). Etched's under-half-voltage claims mark that accounting's limit, though: instruction overhead is only one tax, and a fixed operating point leaves voltage-level co-design on the table (§5). The companion analysis shows a perfect compiler and scheduler brings each substrate to its roofline, and the classic limit studies (Lam & Wilson, ISCA 1992; Wall, ASPLOS 1991) locate the dominant barrier in *control flow and the effort of exposing parallelism* — not in raw data dependence. What separates achieved from achievable is therefore, overwhelmingly, the **effort of specialization** — writing the optimal kernel, building the accelerator, tuning to the platform — and not a wall in the silicon.

Substrate	Penalty vs an ASIC (same function)	Measured by
ASIC	1× — the efficiency ceiling	—
CGRA (word-level reconfigurable)	~2.8× area, on parallel-pattern kernels	Prabhakar et al. (Plasticine), ISCA 2017
FPGA (bit-level reconfigurable)	~20–35× area, ~3–4× delay, ~10–14× power	Kuon & Rose, 2007
GPU (SIMT, software-programmable)	~3–15× energy on regular data-parallel kernels; ~30× perf/W behind a same-era inference ASIC	Chung et al., MICRO 2010; Jouppi et al. (TPU), ISCA 2017
General-purpose CPU	~100–500× energy	Hameed et al., ISCA 2010

Table 1. The measured cost of generality. These are the intrinsic, physical penalties — the realizable frontier of Figure 1 — not what most code actually achieves, which is typically worse because the substrate's potential goes unextracted.

2. The general-purpose vs. custom trade-off is mostly an effort artifact

The trade-off at the center of computer architecture is usually drawn as a Pareto frontier between **flexibility** and **efficiency**: a CPU does anything but wastes energy; an ASIC wastes nothing but does one thing; FPGAs and CGRAs sit in between. The conventional reading is that you *buy* efficiency with flexibility and pick your point on the curve. But that curve is drawn at a fixed, finite *programmer effort* — it is a slice through a three-dimensional surface (flexibility $\times$ efficiency $\times$ effort), and the slice moves as effort changes.

To see why, decompose the efficiency gap between a reconfigurable substrate and an ASIC into two parts. The first is the **intrinsic fabric tax**: the physical overhead of the reconfigurable silicon itself — the switches, the configuration memory, the over-general datapath. This is irreducible; no amount of programmer effort removes it, and it is what the measured ratios of Table 1 capture. The second is §1's **realization gap**: how much of the substrate's *available* efficiency you actually extract, which depends entirely on the quality of mapping, placement, and scheduling — and therefore on effort. At finite effort you extract a fraction; at the limit (the oracle mapping of the companion report) you extract all of it.

The consequence is the whole argument in one picture. As effort rises, the realization gap closes and the achieved-efficiency curve falls toward the realizable frontier — which is just the intrinsic fabric tax. Crucially, **the effort gap is concentrated exactly on the reconfigurable substrates**: it is $\sim$ zero at the custom end (an ASIC has nothing to program), and §1 measured it at one to two orders of magnitude even at the general-purpose end, where compilers are most mature — so it can only be larger in the middle, where the tooling is youngest and the mapping hardest. That concentration is inferred rather than measured — no published study pins the achieved-versus-achievable gap on an FPGA or CGRA, which is part of what Appendix A's experiment is for, and the open CGRA toolchains that do exist are fragmented, routinely failing to map even simple kernels, with no LLVM-equivalent shared infrastructure formed (Walter et al., 2025) — §4's unpooled state, measured in tooling. But the signature is public: reconfigurable fabrics have historically underdelivered not because they are inefficient, but because their efficiency was unreachable at affordable effort. Close that gap — which is what a shared, AI-driven optimization commons does — and a CGRA sits within $\sim 2.8\times$ of an ASIC *while keeping full flexibility* — a tax measured on the regular, parallel-pattern kernels Plasticine was evaluated on; how far it extends across the workload space, especially control-flow-heavy code, is exactly the boundary Figure 5's phase diagram names and Appendix A's simulator sweep is built to measure. The scoping already has one strong answer on record: a composed, programmable architecture — tiny cores with configurable spatial fabric, scratchpads, and DMA — matched the speedups of four diverse domain-specific accelerators (10–150 $\times$ over an out-of-order core) at no more than $\sim 4\times$ the area and power of a single one, under a title that states the thesis outright: *domain specialization is generally unnecessary for accelerators* (Nowatzki, Gangadhar, Sankaralingam & Wright, HPCA 2016; IEEE Micro Top Picks 2017). The line has since been mechanized — accelerators

decompose into a few spatial primitives that tools compose automatically (DSAGEN, ISCA 2020), and generated *programmable* overlays already match untuned fixed-function HLS, reaching 0.55 $\times$ of hand-tuned (the honest residual; OverGen, MICRO 2022) — and independently replicated: for analytics, one programmable systolic design matched a bundle of specialized accelerators (Lottarini et al., ISCA 2019). Sankaralingam co-authored the dark-silicon result that launched the specialization era (Esmaeilzadeh et al., ISCA 2011); the line that started the race concluded the finish was general. The flexibility–efficiency trade-off does not so much shift as **flatten**: under unlimited effort, the only efficiency you still surrender for flexibility is the small, physical fabric tax.